

LLM-Empowered Discovery of Windows APIs Exploitable for Persistent Storage in Fileless Attacks

1st Shuai Cheng
Zhejiang University
Hangzhou, China

2nd Shu Meng
Zhejiang University
Hangzhou, China

3rd Haitao Xu^(✉)
Zhejiang University
Hangzhou, China

4th Shuai Hao
Old Dominion University
Norfolk, VA, USA

5th Meng Han
Zhejiang University
Hangzhou, China

6th Yixin Jiang
Electric Power Research Institute
Guangzhou, China

7th Wenqian Xu
Electric Power Research Institute
Guangzhou, China

Abstract—Fileless attacks have gained significant traction recently due to their minimal footprint and substantial real-world consequences. However, existing methods often trigger detection by antivirus tools when storing malicious content directly in the file systems. In this paper, we propose a novel approach to achieve persistent storage in fileless attacks. This method leverages legitimate Windows Application Programming Interfaces (APIs) to store malicious code either outside the conventional file system or indirectly within it. We established criteria for exploitable APIs and employed ChatGPT for an automated rule-based assessment on 9,410 publicly disclosed Windows APIs. Out of the 80 APIs deemed exploitable, manual verification substantiated the effectiveness of 64, indicating an automation precision rate of 80%. This demonstrates the efficiency of using ChatGPT for automated identification of exploitable Windows APIs. Remarkably, 54 of the 64 APIs are previously unreported for fileless attacks and are typically used in benign programs, posing a challenge in terms of false positives for antivirus software to detect or blacklist them. We developed a malware program using each exploitable API to store and retrieve existing malicious code. During both static and dynamic detection phases, 37 of 64 malware programs successfully circumvented mainstream antivirus solutions. This underscores the efficacy of our persistent storage method in bypassing both static and dynamic detection systems, enabling the storage of arbitrary malicious code on compromised devices. We have communicated our findings to pertinent security vendors and also made the source code for specific Proof of Concepts (PoCs) of the exploitable APIs publicly available.

Index Terms—Vulnerability Analysis, Fileless Attack, Exploitable API, Persistent Storage, LLM

I. INTRODUCTION

In the ongoing cat-and-mouse game in cybersecurity, antivirus software has long been effective in monitoring file system activities and preventing the storage of malicious code within file systems. Consequently, *fileless attacks*—which involve executing malicious activities without relying on files, thus leaving minimal traces in the file system—have emerged as a prominent method for delivering, persisting, and executing malicious code. The term “fileless” does not imply a complete absence of storage, but rather the avoidance of storing malicious executables as conventional files in the file system to evade static detection. For example, an attacker may exploit

a vulnerability to execute a malicious PowerShell command or inject shellcode into a process’s executable memory space, without ever writing a standalone malware file to disk. According to CrowdStrike [10], fileless attack techniques were employed in 8 out of 10 successfully executed breach vectors. Furthermore, a recent Gartner [15] report revealed a 67% year-over-year increase in the number of fileless attack incidents worldwide in 2024.

Red team attackers have developed an array of state-of-the-art techniques customized for each of the three distinct phases of fileless attacks: *delivery*, *persistence*, and *execution* [4]. From the perspective of fileless attackers, both the delivery and execution phases can easily evade mainstream antivirus detection through 0-day/1-day vulnerabilities or techniques like injection [38, 42]. However, the persistence phase, which involves maintaining the shellcode (*i.e.*, malicious code) within the system and loading it into an executable area, becomes the most vulnerable part [47]. In existing persistence techniques, the attacker’s process behavior or binary structure significantly differs from that of benign programs, making it easier for antivirus software to flag them. Consequently, most static and dynamic detection solutions focus on this phase [18, 59].

In this paper, we aim to explore a novel method for achieving highly persistent storage of malicious code, resistant to existing antivirus behavior detection. Our objective is to systematically unveil a comprehensive set of exploitable Windows APIs for the purpose of storing any user input buffer data in persistent storage regions, and then conduct a thorough evasion analysis of these APIs, assessing their effectiveness against prevalent static and dynamic defense solutions. These APIs primarily involve parameters that serve as pointers or handles to memory buffers. When invoked, these APIs prompt the operating system to write the contents of these memory buffers into persistent areas maintained by the system. For example, APIs associated with Windows logging functionality facilitate the writing of log strings into the logging system, enabling retrieval of these logs even after system restarts.

A notable advantage of this storage approach is its dependence on the inherent functionalities of Windows, which is similar to mimicry attacks [57]. Compared to projects like

✉ Corresponding author: Haitao Xu (haitaoxu@zju.edu.cn)

MalAPI.io [37] that catalog a range of Windows APIs with potentially malicious functionalities, the APIs we intend to investigate are typically benign in their usage. These APIs are commonly utilized by ordinary processes and are infrequently linked to known attacks or high-risk malicious activities. From the perspective of antivirus software, this form of persistent storage appears as benign as any application utilizing a functionality provided by Windows. Additionally, Windows typically provides associated retrieval APIs, streamlining the extraction of malicious code. Furthermore, it circumvents direct interactions with the file system, effectively evading the static detection methods employed by antivirus software.

Our methodology involves identifying these exploitable APIs, understanding their specific functions, and crafting corresponding sample code. To comprehensively identify Windows API families capable of enabling persistent storage, we conducted a detailed analysis of the semantics and parameters of all APIs documented in Microsoft’s official documentation [58]. Manual analysis of this magnitude would have been formidable and repetitive. We initially considered developing an automated parser to analyze the structured Microsoft documents. However, these documents do not provide complete information for every API, and it is infeasible to locate keywords in structured documents that can be matched to determine API exploitability. Given that large language models (LLMs) are renowned for their vast knowledge bases [24, 34], we turned to ChatGPT [41] and leveraged its extensive capabilities to streamline the API extraction process effectively by integrating its expertise with Microsoft’s API documentation.

By enriching the documentation with API semantics, parameters, and our criteria for persistent storage as the prompt, we employed ChatGPT for a comprehensive evaluation of all 9,410 Windows APIs. Subsequently, ChatGPT used our most lenient criteria to ensure the comprehensive identification of APIs as potential candidates for persistent storage. Manual verification confirmed the feasibility of each API for storing malicious code. Once these exploitable APIs were identified, we crafted corresponding PoC codes, which includes malicious code and utilizes the specified API to store and retrieve shellcode into persistent regions manipulable through the API. These PoCs were subsequently submitted to VirusTotal [55], a platform offering free analysis of files and URLs for malicious content, for static analysis. Additionally, we subjected them to dynamic testing using 10 cutting-edge antivirus solutions. The results of both static detection and dynamic detection were compared with the baseline.

Through the aforementioned process, we seek to answer the following research questions:

- **RQ1:** What are the characteristics of the APIs exploitable for persistent storage?
- **RQ2:** What prompt engineering techniques are effective for interacting with LLMs to assess APIs?
- **RQ3:** How can we evaluate the performance of LLMs in assessing exploitable APIs?
- **RQ4:** How could we exploit the APIs discovered by LLMs for PoC verification?
- **RQ5:** How effective are these exploitable APIs in evading detection?

In summary, the paper makes the following contributions:

- **Novel Approach for Persistent Storage.** We have developed a novel method to achieve persistent storage of malicious code. Utilizing legitimate Windows APIs, we stored malicious code associated with fileless attacks in areas governed by the Windows operating system, ensuring efficient retrieval and execution.
- **LLM-Empowered API Analysis.** To analyze all 9,410 Windows APIs documented in Microsoft’s official documentation, we employed specific criteria-based prompts to interact with the ChatGPT model. This approach enabled a semi-automated assessment of API semantics and parameters. ChatGPT identified 80 exploitable APIs for persistent storage, and a comprehensive secondary validation process confirmed 64 of these APIs. This highlights the high precision (80%) of the ChatGPT model in aiding the discovery of vulnerable APIs.
- **Systematic Discovery of APIs.** We have developed a comprehensive PoC code system that integrates malicious code with the identified APIs to store and retrieve malicious code. Utilizing this system, we identified 64 APIs exploitable for persistent storage and outlining API names and other details. Our research represents the first systematic exploration of this category of APIs, and we have publicly released the source code for the PoC code system.
- **High Novelty and Evasion Rate.** Through manual inspection, the majority (54 out of 64) of the identified APIs are newly discovered exploitable without being leveraged or documented in blog posts and malware reports. We also assessed the effectiveness of the crafted PoC codes using static and dynamic malware detection solutions, including VirusTotal and various antivirus software. Remarkably, PoC codes corresponding to 37 out of the 64 APIs exhibited low static detection rates (10%) and zero dynamic detection rates (0%). The evasion performance of these APIs is significantly superior to that of traditional persistence technologies, underscoring their potential to effectively bypass antivirus software monitoring on the operating system.

II. BACKGROUND AND RELATED WORK

A. Fileless Attack and Attack Frameworks

The Cyber Kill Chain (CKC) framework [29], developed by Lockheed Martin, divides a cyber attack into seven stages: reconnaissance, weaponization, delivery, exploitation, installation, command and control, and actions on objectives. A typical fileless attack aligns with this framework but highlights its *persistence* capability.

According to MITRE’s ATT&CK framework [2], *persistence* encompasses techniques enabling adversaries to maintain access to systems even though restarts, credential changes, and other disruptions that might interrupt their access. Specifically, once a fileless malware establishes a foothold on the victim

system, achieving persistence necessitates satisfying *two essential requirements*. First, the malware requires an inconspicuous storage mechanism to conceal itself in the system and avoid static detection by antivirus solutions. Second, it must be able to run automatically at startup to endure a system reboot.

B. Prior Work on Persistent Storage Attack and Defense

1) *Registry*: Undoubtedly, the most common mechanism of fileless persistence storage for malicious code on Windows systems is the *Registry*. Processes can customize subkey names, property names, and property values within the Registry. However, research on utilizing the Registry as a persistence storage method is quite antiquated. Apap *et al.* [3] proposed monitoring Registry access from malicious programs as early as 2002. Beyond the Registry, academic research on novel persistence storage methods primarily delves into various other techniques, including code obfuscation, WMI, and Living off the Land (LotL).

2) *Code Obfuscation*: **Attack.** Code obfuscation involves altering the structure of code, essentially transforming an executable program into a semantically equivalent version with entirely different static content [14]. The most commonly adopted method for obfuscation is known as *code packing*. This technique involves changing the initial entry point of the code, encrypting the existing binary instructions, and injecting a block of functional code into the newly established entry point [9]. Upon execution, the functional code at the new entry point decrypts the original binary code and redirects execution back to the original entry point to run the original program. Roundy *et al.* [45] and You *et al.* [62] have documented an extensive range of commercial software and techniques designed for packing. Another prevalent approach is *code compression*. Debray and Evans [11] proposed a method that uses data compression techniques to reduce the size of infrequently executed segments within a program. Obfuscated executables created using this method can overwrite existing code during execution. Popov *et al.* [44] introduced an obfuscation technique that alters the exception handling control flow into signal traps and introduces spurious control flows with redundant instructions when triggered.

Defense. Researchers have explored various techniques for classifying obfuscation and extracting code [8, 12, 21, 60]. Cesare *et al.* [8] employed rapid application-level emulators for unpacking code packages and utilized two types of flow graph matching algorithms for classification purposes. Kawakoya *et al.* [21] proposed an unpacking algorithm based on monitoring memory behavior, which automatically identifies the original code from candidate code by observing changes in accessed memory addresses before and after dump points. Furthermore, antivirus software applies rigorous criteria when evaluating code obfuscation. Research by Zhu *et al.* [60] indicates that benign programs incorporating code obfuscation are highly susceptible to being falsely identified as malicious by antivirus software. This suggests that many antivirus applications do not attempt to deobfuscate and analyze the content of the code;

instead, they categorically classify all obfuscated programs as malicious and proceed to eliminate them.

3) *WMI*: **Attack.** Unlike the Windows Registry, which has been widely exploited for persistent storage of malicious code, WMI is a foundational management technology provided by Windows, serving both local and remote applications. While WMI is primarily utilized for gathering system information, its capability to manipulate the system and its own repository for storing process class information [48] makes it versatile in various stages of fileless attacks, including lateral movement, persistent storage, and code execution [1]. Graeber [16] and Maduranga [32] have presented methods leveraging WMI to write arbitrary content into a WMI-maintained database. This approach effectively bypasses file system storage while achieving long-term storage of shellcode.

Defense. Research on detecting WMI attacks generally revolves around monitoring the behavior of processes [27, 50]. It is important to note that benign software typically does not interact with WMI. Therefore, when a process without codesigning frequently interacts with WMI, it is typically flagged as malicious. Researchers like Tanana and Dmitry [50] and Liu *et al.* [27] conducted research based on a process behavior model to detect malicious operations in WMI. Moreover, antivirus software typically monitors every interaction between processes and WMI, alerting users to such actions and even blocking a process's connection to WMI [53].

4) *LotL*: **Attack.** LotL is a contemporary attack technique that exploits native executable files in the Windows system, equipped with functionalities like code execution, file duplication, and other sensitive operations. Attackers leverage these capabilities to embed malicious payloads within legitimate process contexts, effectively evading detection by antivirus software. Li *et al.* [25] and Pontiroli *et al.* [43] discussed the use of PowerShell scripts for LotL attacks. Barr-Smith *et al.* [6] conducted a comprehensive analysis of LotL attacks, assessing their prevalence in Advanced Persistent Threats (APTs) [17, 61] and their ability to evade detection by antivirus solutions. Efforts to identify LotL binaries (LOLBins) have resulted in LOLBAS project [30], which outlines all native binary files in Windows system exploitable for LotL attacks. The project provides specific LOLBin parameters for attacker and categorizes LOLBins into various groups, including downloading, execution, copying, application whitelist bypassing, and certification.

Defense. Extensive research has been conducted on LotL attacks involving scripts [26, 46, 56]. Rusak *et al.* [46] proposed a hybrid method that combines traditional program analysis like Abstract Syntax Trees with deep learning to detect malicious PowerShell scripts. Wael *et al.* [56] utilized a combination of machine learning and static script analysis to detect malicious VBScript samples. The tools listed in LOLBAS have been the subject of research conducted by Ongun *et al.* [39] and Stamp *et al.* [49]. Mainstream antivirus solutions preemptively flag and intercept all processes listed in LOLBAS, rendering most LOLBins unusable for attacks.

III. EXPLOITABLE API DISCOVERY METHODOLOGY

We propose a novel approach to achieve persistent storage of malicious code outside or indirectly within the file system by leveraging legitimate Windows APIs.

A. Threat Model

As illustrated in Figure 1, we contextualize the attack process within the framework of the CKC model [29]. For simplicity, we divide this process of fileless attack into three distinct phases:

- **Delivery:** This encompasses the *Reconnaissance*, *Weaponization*, *Delivery*, and *Exploitation* stages of the CKC model. In this phase, the attacker crafts their weapon, exploits vulnerabilities to penetrate the victim’s computer, gains control permissions, and delivers the malicious payload onto the victim’s machine.
- **Persistence:** This aligns with the *Installation* stage of the CKC model. In this phase, the attacker aims to persistently store malicious code within the system and ensure its ability to auto-start, thereby enabling malicious code execution.
- **Execution:** This phase includes the *Command & Control* and *Actions on Objectives* stages of the CKC model. The attacker executes shellcode to achieve further objectives such as privilege escalation, information extraction, lateral movement and others.

Given the aforementioned threat model, our research assumes that the attacker has already completed the *Delivery* phase without being detected by antivirus software. Our study then narrows its focus on exploring methods for achieving persistent storage of malicious code **outside or indirectly within** the file system without being detected from the antivirus solutions. Our approach and experiment will exclude the auto-start component. For research concerning the auto-start, reference can be made to prior works [40, 51, 52].

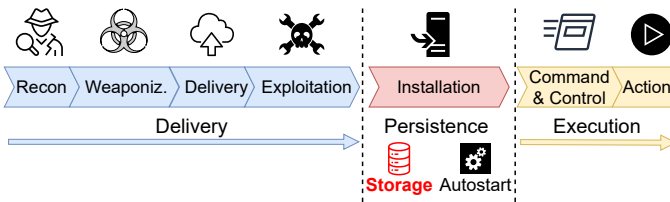


Fig. 1: Our focused stage among the 7-step CKC model

B. RQ1: Criteria for Exploitable APIs

The criteria for exploitable APIs should ensure that compliant APIs can be used to store arbitrary shellcode in a non-filesystem location, with a mechanism to retrieve it after a system reboot. Therefore, given a buffer containing customized malicious code, a legitimate and exploitable API for persistent storage should adhere to the following criteria:

- The API should have a function parameter that is a pointer referencing the specified buffer, with its input modifier designated as [in], indicating that the API reads from the buffer. To facilitate the writing of arbitrary shellcode, any

restrictions on the content of the buffer should not impede the writing of arbitrary characters.

- The API should be capable of reading from the specified buffer and writing its content to a persistent region within the victim’s system for the shellcode storage. A persistent region refers to a section which remains unaltered (*i.e.*, without being cleared or overwritten) after system reboots. This region should not reside directly within the file system (*e.g.*, unlike the Registry or WMI, which indirectly write contents to the file system).
- In cases that shellcode in the buffer is too long, requiring multiple invocations of the API for reading and writing to storage regions, the API should not overwrite previous storage sections during subsequent writing operations.
- After the API writes malicious code into a persistent region, there should exist other APIs or mechanisms that can retrieve the code from the persistent region for shellcode execution. The API also needs to be compatible with the latest Windows 10 system.

Our objective is to identify a comprehensive set of Windows APIs that meet these criteria and develop a new adversarial attack that can persistently plant malicious code using these APIs with exploitable and user-customizable buffers. This new attack is expected to easily evade detection by mainstream antivirus solutions since, from their perspective, it appears as though the API’s intended function is executed, rather than storing malicious code.

It is important to note that not all APIs meeting these criteria are entirely novel. For example, those related to writing to the Registry have been used in prior attacks [22]. However, for the systematic exploration of these exploitable APIs, a complete list will be provided in §V-F, along with details identifying which have been previously exploited and which represent novel instances.

C. Exploitable API Identification Approach

To identify as many APIs aligning with the criteria above as possible, we designed the following approach, illustrated in Fig. 2.

- Extraction of Windows APIs:** We started with extracting 9,410 Windows APIs listed on Microsoft’s official site [58], and structured their corresponding documentations using a standardized JSON format (§IV-A).
- LLM-based Vetting of APIs:** We converted the criteria (§III-B) into prompts and submitted these prompts, along with each API’s JSON documentation, to ChatGPT (§IV-B), in order to vet all APIs for their potential usage in persistently storing shellcode.
- Manual Verification and PoC Implementation:** APIs that were evaluated as feasible by ChatGPT underwent rigorous manual review. We crafted corresponding PoCs for the APIs that were reaffirmed as feasible and flagged any APIs deemed infeasible after manual inspection. This process resulted in a list of exploitable APIs. We elaborate the manual verification process in §IV-C and the PoC implementation details in §V-B.

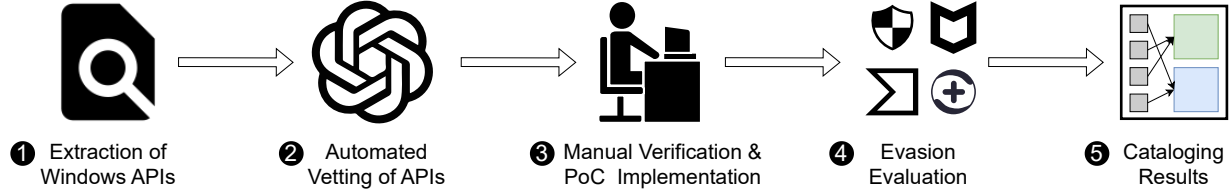


Fig. 2: Procedure of exploitable API identification

- (iv) **Evasion Evaluation:** We conducted both static and dynamic assessments of the implemented PoCs. For static analysis, we used VirusTotal to test the PoCs with various antivirus software. For dynamic analysis, we fortified a Windows 10 environment with several widely-used antivirus software solutions and ran the PoC programs. Our focus was to determine if the act of storing or retrieving malicious code triggered security alerts from antivirus software. A holistic overview of the evaluation setup is provided in §V-C. The evasion results and their comparison with baseline are provided in §V-E.
- (v) **Cataloging Results:** For each API, we documented relevant information, including pertinent function parameters, required privileges, and any auxiliary constraints. We discussed the storage location and the novelty of these exploitable APIs in §V-F.

This method enables systematic identification of exploitable APIs and assessment of their evasion effectiveness.

IV. GPT-POWERED DISCOVERY OF EXPLOITABLE APIS

A. Formatting Windows APIs for ChatGPT Input

We compiled a comprehensive list of 9,410 Windows APIs sourced from the Microsoft Win32 API reference documentation [58]. For each API, we retrieved essential information, including its function name, function prototype, parameters, and a detailed description of its intended purpose. This data was organized in the following format, using the “StrCmpW” function as an illustrative example:

```
{
  "Name": "StrCmpW",
  "Prototype": "int StrCmpW([in] PCWSTR psz1, [in] PCWSTR psz2);",
  "Parameter": "[in] psz1 Type: PCTSTR. A pointer to the first null-terminated string to be compared. [in] psz2 Type: PCTSTR. A pointer to the second null-terminated string to be compared.",
  "Description": "Compares two strings to determine if they are the same. The comparison is case-sensitive."
}
```

As illustrated in this example, “Name” denotes the function’s name, while “Prototype” and “Parameter” directly align with the “Syntax” and “Parameter” sections in Microsoft’s structured documentation, respectively. The “Description” section encapsulates a concise overview of the function’s capabilities, along with other descriptions from the documentation, such as the “Remarks” section, where critical details of the storage location are provided.

B. RQ2: Prompt Design for ChatGPT

When presented with a Windows API in the specified format, we leveraged GPT-4 APIs to evaluate the potential exploitation of the API for persistent storage. This evaluation requires specific prompts for ensuring accurate assessments.

Challenge of Applying GPT. Simply relying on GPT’s knowledge base often fails to yield satisfactory results. Specifically, its performance is influenced by two key factors: the popularity of APIs on the Internet (*i.e.* discussed online frequently or not) and the comprehensiveness of Microsoft documentation on a given API. When an API is popular, GPT generally produces accurate judgments, even without supplementary Microsoft documentation. When the API is less well-known and lack detailed Microsoft documentation, GPT frequently generates inaccurate outputs. In such instances, we must iteratively refine our prompts and provide sufficient domain knowledge about the API.

Initially, we directly applied the criteria outlined in §III-B. However, when these criteria were utilized solely as prompts into GPT-4, two key issues arose in this process:

- GPT-4 may erroneously identify APIs with names containing terms like “global” (*e.g.*, *GlobalAddAtomA*) as meeting the criteria for persistent storage, even when they do not.
- The model has a tendency to focus solely on the function’s intended purpose, potentially overlooking its latent capabilities that go beyond its primary design.

These issues led to an initial assessment of GPT-4, revealing 124 APIs, with only 35 undergoing manual verification as exploitable, resulting in a precision rate of merely 28.2%. Some known exploitable APIs were also found to be ineffective, such as *SHRegSetUSValueA*, which writes contents into the Registry. Consequently, we refined the prompt by placing specific emphasis to address the above two issues, highlighting that certain areas, which might be misconceived as persistent regions, are in fact not, and also emphasizing the need to disregard the intended purpose of the function. The resulting final prompt, which has demonstrated success in yielding the desired outcomes, is given as follows:

“Upon receiving a JSON input containing information about a Windows API function, your task is **two-fold**: **1. Understanding and Analysis:** First, understand and analyze the function contained within the JSON. This information encompasses the function’s name, prototype, description, and detailed description of its parameters. You need to integrate the content from the JSON with your knowledge about the specific function. **2. Evaluation:** Subsequently, determine if the

function can write the value of a memory object referenced by a pointer parameter annotated as [in], or associated with a handle parameter annotated as [in], to a system region. This region must **satisfy the following criteria**:

- The data in the region remains **retrievable** after a system reboot.
- **The region will not be automatically cleared or reset** after system reboot. For example, global cache, global memory, or global atomic table are not acceptable.
- The region cannot reside directly within the file system (unlike **Registry / WMI**, which are acceptable).
- The written value in the region should be **a value that could be customizable**, rather than predefined.
- If the buffer corresponding to the function parameter has a finite length, **the region should allow subsequent storage operations without overwriting previous ones** when the API needs to be invoked multiple times.
- The API needs to be compatible with the Windows 10 system.

*Please disregard the original design intent or purpose of the function, and focus solely on the possible outcomes. If the function possesses the capability of writing into a system region that meets the above criteria, **respond with "Yes"**. If the function lacks this capability, **respond with "No"**. Refrain from providing any supplementary text or explanations."*

C. RQ3: Manual Inspection of GPT-4's Response

True Positives and Precision. Using the prompt mentioned earlier, we conducted a mass feasibility assessment of all 9,410 Windows APIs. Out of these, 80 Windows APIs were initially identified by GPT-4 as suitable for persistent storage. To ensure the precision of this assessment, we manually created PoC implementation for each API, attempting to store malicious code, and found that 64 out of the 80 APIs (80%) were confirmed as functional. Detailed information about these 64 APIs is presented in §V-F.

False Positives. There were a total of 16 APIs that were falsely flagged as potential candidates by GPT-4 (i.e., false positives). Subsequently, we conducted a comprehensive analysis to discern the reasons behind the false flagging of these 16 APIs and to identify the specific criteria outlined in §III-B that they violated. The outcomes of this analysis are succinctly summarized in Table I. It is evident that the majority of false positives (FP) occurred due to GPT-4's misconceptions about the concept of persistence or its inability to identify if a function parameter is constrained to specific values.

False Negatives. Comprehensively assessing False Negatives (FN) in the evaluation of GPT-4 poses challenges due to the impracticality of manually verifying all 9,410 APIs. To address this, we propose two methods to measure the performance.

- **Results of Random Sampling.** We conducted a random sampling of 200 APIs from the 9,330 identified by GPT-4 as non-exploitable. Manual assessment against the criteria outlined in §III-B revealed that only one API, specifically `NetServerSetInfo` [35]—utilized for initializing operation parameters on a remote server—met all the criteria,

TABLE I: List of false positive APIs and the criterion violated

API Name	Reason for FP	Criterion Violated
CertOpenStore	Unable to persist after restart	(ii)
CertSetStoreProperty	Unable to persist after restart	(ii)
ChangeDisplaySettingsA(W)	Reject arbitrary user inputs	(i)
ChangeDisplaySettingsEx	Reject arbitrary user inputs	(i)
CryptProtectData	Unable to persist after restart	(ii)
HttpSetServiceConfiguration	Unable to persist after restart	(ii)
SetNtmsObjectAttributeA(W)	No longer used after Windows 7	(iv)
SetVolumeLabelA(W)	Limited buffer length	(iii)
SHRegSetValue	No longer used after Windows 7	(iv)
SLInstallLicense	Reject arbitrary user inputs	(i)
SLInstallProofOfPurchaseEx	Reject arbitrary user inputs	(i)
StgCreateStorageEx	Write file in plaintext	(ii)
WinBioSetCredential	Reject arbitrary user inputs	(i)

constituting 0.5% of the sample. This particular API may have been overlooked because it requires both specific parameter settings and elevated permissions, restricted to members of the local administrator or server operator group. Additionally, some APIs partially met the criteria, such as the `SCardWriteCacheA(W)`¹, employed for writing smart card attributes into a global cache maintained by the smart card resource manager. Although seeming to fulfill our requirements, further investigation revealed that the global cache is cleared upon system restart. This highlights GPT-4's adherence to the criteria in §IV-B that "The region will not be automatically cleared or reset."

- **Analysis of Security Reports.** To identify potential persistence API vectors, we conducted searches on Google, Github, and security websites such as *Securelist* [20] using keywords like "Windows API" and "code storage." Notably, GPT-4 accurately identified all APIs mentioned on these platforms as exploitable, as introduced in §V-F2, which means no false negatives were found from security reports.

According to the evaluations above, it is evident that the FN rate in GPT-4's judgments has been significantly minimized to a mere 0.5%, mainly due to the lenient criteria in the prompt.

D. On the Stability and Generalizability of Our GPT-based API Analysis

1) *Generalization Across Different LLMs:* To assess the stability and generalizability of our LLM-based API analysis, we extended our evaluation beyond GPT-4 to other mainstream LLMs. Based on the LMArena Team leaderboard ranking [28], we selected the top ten LLMs and applied the same API criteria and prompts described in §IV-B. To ensure compatibility, we included only the highest-ranked model from each provider and excluded reasoning-specialized models.

The results, presented in Table II, demonstrate high consistency across mainstream LLMs. Precision across these models ranges from 72.62% to 86.49%, which is consistent with the 80% precision achieved by GPT-4 in our original evaluation. This consistency indicates that our rule-based prompt design and exploitability criteria are not tied to a specific LLM implementation and generalize effectively across different LLMs.

A(W) denotes both ANSI and Unicode versions of a function

TABLE II: Stability of Exploitable API Identification Across Different LLMs

LLMs	APIs Identified	TP	Precision
claude-opus-4-6	75	64	85.33%
gemin-3.1-pro-preview	78	64	82.05%
gpt-5.2-chat-latest	74	64	86.49%
dola-seed-2.0	76	64	84.21%
grok-4.1	76	63	82.89%
glm-5	79	63	79.75%
ernie-5.0-0110	82	62	75.61%
kimi-k2.5	80	62	77.50%
qwen3.5-397b-a17b	82	63	76.83%
deepseek-v3.2-exp	84	61	72.62%

2) *Repeated Trials and Reproducibility*: To further evaluate stability over time, we repeated the full API assessment process three times for each LLM under identical experimental settings. Across all models, the outputs remained unchanged in terms of the TP count. This result indicates that, under controlled prompt configurations and deterministic inference settings, the LLM-based API evaluation process demonstrates strong reproducibility. Therefore, the empirical findings reported in this study are stable and not attributable to stochastic variation in model responses.

V. PoC CODE SYSTEM AND EVASION EVALUATION

A. Overview

In this section, we conducted an in-depth examination of the 64 exploitable APIs identified by GPT-4, as discussed in §IV-C. The examination involved categorizing these APIs based on their functionalities, listing each API within its respective category, specifying the required privileges for each API, and highlighting the parameters that could be exploited for persistent storage. Furthermore, we clarified the specific areas where these APIs stored malicious data.

As part of the examination, we also created PoC code that utilized these APIs. After compiling the code into executable files, we subjected them to static binary analysis to assess the extent to which the 64 APIs could evade static detection. Subsequently, for dynamic analysis, these executables were executed in real Windows systems equipped with prevalent dynamic detection solutions to determine if their behaviors triggered security alerts as malicious.

The composition of the PoC code system is detailed in §V-B. The configurations for both static and dynamic testing environments are described in §V-C, while the baseline setup is outlined in §V-D. The characteristics of verified exploitable APIs are presented in §V-F. A summary of the evasion results for these APIs is provided in §V-E, and an in-depth analysis of why antivirus software did or did not detect these PoCs is discussed in §V-G.

B. RQ4: PoC Code System Design

The procedure for crafting PoC code, as depicted in Figure 3, involves several key components:

- (1) **Malicious Executable for Encoding**: Any shellcode written in C language can be utilized. We choose some

malicious executable such as `mimikatz` (A password probing tool), `cobalt strike` (a remote access trojan tool), and `Metasploit` (a penetration and remote access tool). These tool are typically detectable and can be neutralized by security software. For example, the x64 version of `mimikatz` exhibited a high detection rate of 56/72 on VirusTotal. We utilized `donut`, a shellcode generation tool, to transform the executable of `mimikatz` to shellcode without `\0` in this stage to prevent string truncation. We have tested and confirmed that the shellcode still has a high detection rate on VirusTotal, similar to the original executable.

- (2) **Encoded Data and Decoding Module**: This module is designed to encode the shellcode of malicious programs and embed it into the PoC processes. We developed a *decoding module* to revert the encoded shellcode back to its original form. This ensures that the encoded shellcode remains undetected before storage and execution. Once decoded and invoked by the exploitable APIs, only the sensitivity level of the APIs for antivirus solutions would determine the success of evasion.
- (3) **Writing Module and Reading Module**: The *writing module* divides the decoded shellcode into segments and passes each segment as a parameter to the identified exploitable APIs. The *reading module* retrieves the shellcode written by the writing module. It sequentially reads the segment via APIs and assembles these chunks in memory to form the complete shellcode. The size of chunks are adjusted accordingly to the the storage capacity of each API for single instances, *e.g.*, 64, 256, or 512.

The entire compilation process was executed on a Windows 10 operating system using the Visual Studio 2022 v143 toolset. The PoC executable was statically compiled in x64 release mode. The encoding data and decoding module were combined with the writing module to create the PoC samples for writing shellcode (abbreviated as *PoC-W*), while the reading module was compiled to create the PoC samples for reading shellcode (abbreviated as *PoC-R*). We have publicly released the source code for certain PoCs of the exploitable APIs on Zenodo, accessible via the link [63].

C. Detection Environment Setup

Static Detection: For static detection, we uploaded the compiled PoC-W and PoC-R to the VirusTotal platform, which conducts analyses using inputs from 72 renowned security vendors, including *Cylance*, *Kingsoft*, *McAfee*, *Microsoft*, *Symantec*, and others. We recorded the detection rate for each API and collected information on the threat categories and family labels provided by VirusTotal.

Dynamic Detection: Our dynamic detection tests were performed on 10 physical machines, all running Windows 10 Professional Version 22H2 with administrative privileges. We selected these antivirus products based on the 2023 antivirus software testing report [5] by AV-Comparatives, an international independent security testing organization. The chosen

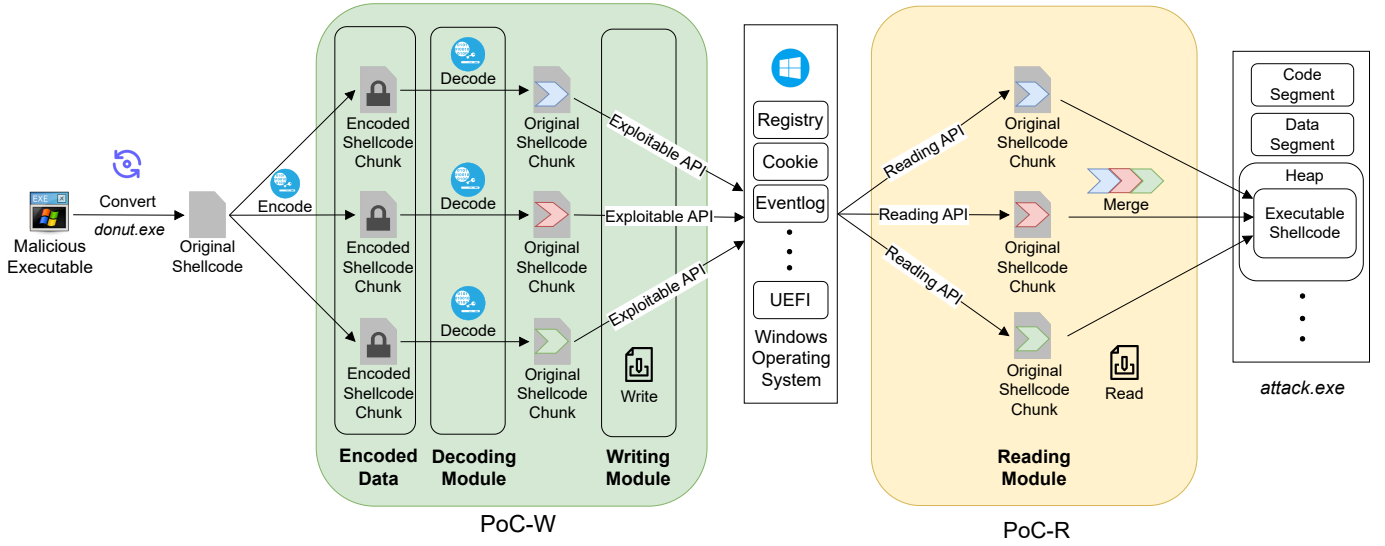


Fig. 3: Procedure of PoC code system Composition

products are *Avast*, *AVG*, *Bitdefender*, *ESET*, *G Data*, *Kaspersky*, *McAfee*, *Microsoft Defender*, *TotalAV*, and *Total Defense*. These antivirus products employ both static and dynamic detection techniques and routinely scan process memory. Moreover, some products incorporate additional sensitivity markers for LotL processes [6] (e.g., Kaspersky) and imposes stricter monitoring on registry activities (e.g., Bitdefender).

Here is how the dynamic detection tests were conducted:

- (1) We initially executed PoC-W to store the shellcode.
- (2) Then, the machine and antivirus software were rebooted.
- (3) After the reboot, we ran PoC-R to retrieve and assemble the shellcode in memory.
- (4) Just before the shellcode was set to execute (the memory has been allocated with an executable flag and filled), we introduced a breakpoint using the `getchar` function.

If, before reaching this breakpoint, the antivirus software neither deleted the PoC-W and PoC-R files, nor terminated the PoC-W and PoC-R processes, nor reported any malicious behavior, then we considered PoC-W and PoC-R to have successfully evaded dynamic detection. Otherwise, we recorded the respective detection logs. Finally, we combined the results to determine the success rate of evading.

D. Baseline Setup

The four persistent storage techniques outlined in §II-B serve as the baseline for evaluating the evasion of exploitable APIs. For each technique, we have selected three classical implementations, and the following details our selections:

- **Registry:** We used Windows' Registry APIs to store malicious code in three registry entries: subkey names, attribute names, and attribute data values.
- **WMI:** We employed three different methods recommended by Microsoft [36] to invoke WMI for storing malicious code within the attribute of a specific class. These methods included Powershell, Wmic.exe, and .NET interfaces [31].
- **Code Obfuscation:** We subjected malicious executable code to obfuscation using three well-known packing tools: UPX,

VMProtect, and PECompact. These tools incorporate both code packing and compression techniques.

- **LotL:** We crafted three different tools as mentioned in §II-B4: malicious VB Scripts (executed via the system's built-in `Wscript.exe`), malicious PowerShell scripts, and an executable that utilizes `certutil.exe` to download and encode shellcode into local files [13].

We utilized these 12 distinct techniques for the persistent storage and retrieval of shellcode as a baseline, incorporating both static and dynamic detection methods across these two phases. The specific shellcode system employed by these techniques is identical to that described in §V-B. These techniques are categorized into Baseline-W and Baseline-R, analogous to PoC-W and PoC-R, based on the shellcode's reading and writing modules. The environments used for both static and dynamic detection are the same as those outlined in §V-C.

E. RQ5: Evaluation Results

We have completed static and dynamic testing of 64 APIs and 12 baseline implementations. The dynamic and static detection result of 12 baseline implementations are enumerated in Table III. Full evaluation results of 64 APIs can be accessed in the Table IV in Appendix.

Evaluation Results by Comparison with Baseline: We used box plots to illustrate the distribution of the results from static and dynamic detection, as depicted in Figure 4. The figure clearly illustrates that POC's detection evasion performance exceeds that of the baseline in most cases. This is evidenced by the fact that the two boxes representing the POC are significantly lower on the y-axis compared to the two representing the baseline. A lower detection rate in the figure indicates a more advanced ability to evade detection.

In the static detection graph, the median and mean values for POC-R and POC-W detections are significantly lower than those for Baseline-R and Baseline-W. In the dynamic detection graph, most PoC-R evaded dynamic detection by all 10 antivirus software and the vast majority of POC-W dynamic

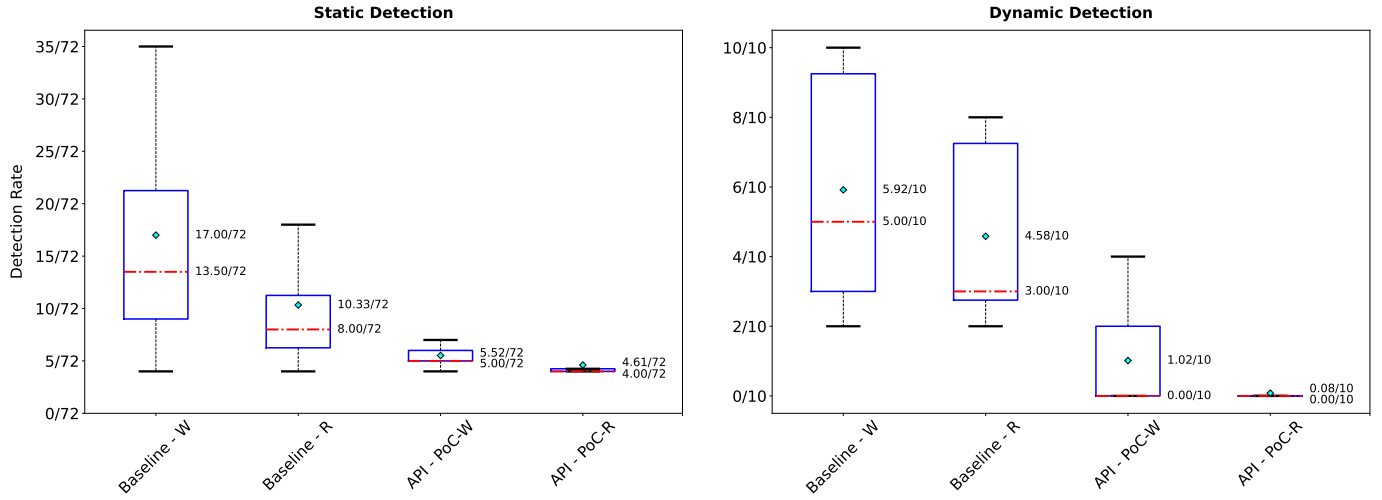


Fig. 4: Box plot of static and dynamic detection results

detection rates do not exceed 2/10, whereas all baseline-R and baseline-W were dynamically detected by at least 2 antivirus software. This clearly demonstrates that the evasion capability of these exploitable APIs is higher than the baseline.

Evaluation Results Based on Custom Criteria: Based on the results of static and dynamic detection for all 64 exploitable APIs we identified, we established a set of criteria for determining the effectiveness of an API in evading both static and dynamic detection. An API is deemed successful in evading the detection if it meets all the following conditions:

- **Static Detection:** Considering that certain antivirus engines on VirusTotal may have limited capability to conduct deep analysis of samples and may abuse AI technology to boost detection rates [60], causing erroneous identification and marking of programs based on specific code sequences, it is impractical to anticipate complete evasion of all 72 antivirus engines on VirusTotal. According to an assessment report from VirusTotal [54], when the detection rate surpasses 15%, the sample can be deemed suspicious. Adhering to more stringent criteria, we have revised the baseline for the detection rate to 10% (i.e., 7/72), and the API should not be flagged with any malware family labels.
- **Dynamic Detection:** During dynamic evasion analysis, the PoCs of an API should not be identified by any of the ten cutting-edge dynamic detection solutions.
- Both PoC-R and PoC-W must satisfy the above conditions.

Out of the 64 APIs tested, a total of 37 APIs met the above conditions, which indicates that 57.8% of the exploitable APIs we identified can effectively bypass majority of mainstream antivirus software. This also demonstrates the effectiveness of using legitimate APIs for implementing persistent storage in terms of conducting attacks and evading detection.

F. Characteristics of Verified Exploitable APIs

We enumerated 64 APIs that underwent manual verification and satisfied the defined exploitability criteria, as detailed in the Table IV in Appendix. Each API entry includes its functional description, corresponding persistent storage areas,

and assessed level of novelty. This section focuses on analyzing the storage locations and novelty levels, while additional API characteristics—such as available parameters, required permissions, and storage capacity results—are also provided in the Table IV in Appendix.

1) *Storage Location: Registry or Alternate Areas:* Among the 64 APIs analyzed, 44 store shellcode in the Windows Registry. APIs prefixed with “SH*” directly manipulate the registry, while others writes Registry indirectly. Although their underlying function implementations are similar, the APIs which indirectly access the Registry typically have predefined entry names or path prefixes, avoiding the creation of custom registry paths. Consequently, antivirus software often deems such indirect registry access harmless, incapable of disrupting system-related registry behaviors. The remaining 20 APIs utilize different persistent areas for storage. APIs associated with credentials and cookies store data in databases maintained by Windows. The WritePrivateProfileStructA(W) and NCrypt* APIs store data in encrypted files, similar to the log files created by ReportEventA(W). Additionally, Firmware-related APIs store data in UEFI firmware. These distinct storage areas, some outside the file system and others indirectly within it, avoid direct interaction with the file system. This approach aids in evading the static detection from antivirus software.

2) *Novelty of Exploitable APIs:* To determine if the identified APIs have been employed by other attackers or referenced in security reports, we conducted extensive searches on GitHub, Google Search, Google Scholar and famous security websites. This involved using the API names along with keywords like “fileless attack” and “storage” in our queries. Among the 64 APIs, 10 were detected in existing fileless malware reports. Notably, Kaspersky [23] reported a significant fileless attack based on the ReportEventA(W) API in 2022, offering a detailed sample analysis. Reports dating back to 2014 and 2021 [22, 33] highlight Registry API attacks, indicating the established usage of direct registry manipulation for persistent storage. Furthermore, a GitHub blog post in 2023 [19] discussed the utilization of UEFI

TABLE III: List of detection results for baselines (R means Baseline-R and W means Baseline-W)

Technique Name	Technique Type	Static Detection		Dynamic Detection	
		Rate	Label	Rate	Detected by
Stored in subkey names through Registry APIs	Registry	4/72(W) 4/72(R)	None(W) None(R)	2/10(W) 2/10(R)	Bitdefender and Kaspersky(W) Bitdefender and Kaspersky(R)
Stored in attribute names through Registry APIs	Registry	6/72(W) 4/72(R)	None(W) None(R)	2/10(W) 2/10(R)	Bitdefender and McAfee(W) Bitdefender and McAfee(R)
Stored in attribute data values through Registry APIs	Registry	5/72(W) 4/72(R)	None(W) None(R)	3/10(W) 3/10(R)	Bitdefender and etc. (W) Bitdefender and etc. (R)
Stored in WMI through Powershell	WMI	15/72(W) 8/72(R)	<i>trojan.baxter/powershell(W)</i> None(R)	5/10(W) 3/10(R)	McAfee and etc. (W) McAfee and etc. (R)
Stored in WMI through Wmic.exe	WMI	10/72(W) 8/72(R)	None(W) None(R)	5/10(W) 3/10(R)	TotalAV and etc. (W) TotalAV and etc. (R)
Stored in WMI through .NET interfaces	WMI	10/72(W) 8/72(R)	None(W) None(R)	3/10(W) 3/10(R)	G Data and etc. (W) G Data and etc. (R)
Obfuscated malicious VBS scripts	LotL	28/72(W) 15/72(R)	<i>trojan.VBS.Agent.ANN(W)</i> <i>trojan.VBS.Agent.ANN(R)</i>	9/10(W) 6/10(R)	Kaspersky and etc. (W) Kaspersky and etc. (R)
Obfuscated malicious PowerShell scripts	LotL	15/72(W) 10/72(R)	<i>trojan.baxter/powershell(W)</i> <i>trojan.baxter/powershell(R)</i>	10/10(W) 8/10(R)	All solutions(W) McAfee and etc. (R)
Using certutil.exe to download and encode shell-code	LotL	12/72(W) 7/72(R)	None(W) None(R)	3/10(W) 2/10(R)	Kaspersky and etc. (W) Kaspersky and etc. (R)
Code obfuscation using UPX	Code Obfuscation	45/72(W) 28/72(R)	<i>trojan.mimikatz/marte(W)</i> <i>trojan.mimikatz/marte(R)</i>	10/10(W) 8/10(R)	All solutions(W) Kaspersky and etc. (R)
Code obfuscation using VMProtect	Code Obfuscation	19/72(W) 10/72(R)	<i>Win32/Packed.VMProtect.AB(W)</i> <i>Win32/Packed.VMProtect.AB(R)</i>	10/10(W) 8/10(R)	All solutions(W) McAfee and etc. (R)
Code obfuscation using PECompact	Code Obfuscation	35/72(W) 18/72(R)	<i>trojan.mimikatz/marte(W)</i> <i>trojan.mimikatz/marte(R)</i>	9/10(W) 7/10(R)	Total Defense and etc. (W) Total Defense and etc. (R)

firmware APIs for variable storage, aligning closely with our findings. Conversely, the remaining 54 APIs were not located in fileless malware reports, or if mentioned, they were utilized not for persistent storage purposes. Hence, we classify these 54 APIs as novel.

G. Speculations on the PoC's Evasion and Detection

Detection Analysis. In our previous evaluation, the PoCs of some APIs were statically or dynamically detected by antivirus software, which mainly stem from two reasons:

- **Monitoring of Sensitive Registry Entries.** Some antivirus solutions demonstrate good performance in both dynamic and static detection of APIs that directly manipulate the registry. However, their dynamic detection falls short when APIs indirectly write to the registry. Our speculation is that they monitors specific sensitive registry keys and paths. Direct registry entries are often sensitive custom paths, while indirect entries typically involve predefined fixed items. When these items are unrelated to startup and system configuration, they are deemed safe.
- **Sensitivity of API Functionality.** Beyond registry manipulation, other APIs with high detection rates often possess sensitive functionalities. `CredUIStoreSSOCredW` manages the system's stored single sign-on credentials, and `SetupSetSourceListA(W)` configures the system's installation source list. These APIs are infrequently invoked by benign programs, making their invocation by unsigned processes more likely to be directly blocked by antivirus software. Notably, the PoC-W often exhibits a higher detection rate than PoC-R, reflecting the heightened sensitivity associated with writing operations.

Evasion Analysis. For APIs with low static detection rates and without dynamic detection, our analysis reveals that their evasion can be attributed to the following two factors:

- **Similarity to Benign Behavior.** Many APIs that have evaded our detection are commonly invoked by benign processes, such as `NCryptCreatePersistedKey` for cryptographic key creation and `ReportEventA(W)` for log writing. Execution sequences involving these API invocation, memory allocation, and then execution are prevalent in benign processes. This commonality poses a challenge for antivirus software, as monitoring such sequences could result in numerous false positives. One effective way to distinguish such behavior is to monitor the contents of API parameters, but the high frequency of these API calls leads to significant performance degradation in parameter monitoring.
- **Different Focus on Parameters.** Some APIs are monitored by antivirus software due to their sensitive functionality. However, in the process of persistent storage, the primary monitored parameters exhibited no anomalies, whereas the actual storage occurred through alternative parameters. For instance, as for the `CreateServiceA(W)` function, antivirus software primarily focuses on monitoring the process path and command line parameters. However, we could store the shellcode in the `lpDisplayName`, effectively bypassing detection. This evasion tactic arises from antivirus software's excessive focus on the API's intended function, overlooking other potential features.

VI. MITIGATION

The APIs identified in this study often serve legitimate system functions, making their abuse difficult to detect through isolated API monitoring. Effective mitigation requires contextual and behavioral analysis that spans multiple stages of an attack. We outline several complementary directions below.

- **Parameter-Level Inspection.** Security products should inspect API parameters for anomalous inputs. Abuse often involves writing attacker-controlled buffers to persistent storage, indicators such as high-entropy, binary, or unusually

long inputs may signal misuse. In addition, large payloads require repeated API calls, making anomalous write frequencies to the same location another behavioral indicator [59].

- **Cross-Stage Behavioral Correlation.** Analyzing persistence-related APIs in isolation is insufficient for reliable detection. A more robust approach correlates behaviors across multiple stages within the same process. For example, the sequential or co-occurring appearance of (i) writes to persistent storage via the APIs studied, (ii) memory allocation with executable permissions (e.g., `VirtualAlloc` with `PAGE_EXECUTE`), and (iii) execution mechanisms (e.g., function pointer calls or process injection) provides a stronger signal of malicious intent. Integrating such execution-stage monitoring would likely increase detection rates beyond those achievable by analyzing the storage phase alone.
- **Monitoring the Storage-to-Execution Transition.** Modern endpoint detection and response (EDR) systems routinely scan heap and executable memory regions for shellcode signatures. While our study focuses on the storage phase, enhanced monitoring of the retrieval of data from persistent regions and its subsequent mapping into executable memory provides an effective detection pivot point, especially when correlated with preceding persistence-related API activity.
- **API Hooking and Input Validation.** Security vendors may implement hooks for high-risk APIs to enforce runtime input validation. Inputs that exceed expected semantic bounds or deviate from an API's intended data structure can be sandboxed, logged for further analysis, or rejected before being committed to persistent storage.
- **Cross-Platform Applicability.** While our study focuses on Windows, analogous persistence mechanisms exist on Linux (e.g., `systemd` services and cron jobs) and macOS (e.g., launch agents, property list files, and keychain storage). The mitigation principles outlined above, including parameter inspection, behavioral correlation, and memory-execution linkage, are therefore applicable across operating systems.

In summary, effective mitigation should integrate parameter-aware monitoring, analysis of repeated write patterns, cross-stage behavioral correlation, and tracking of the transition from storage to memory execution. A defense strategy that correlates signals across multiple attack stages will be significantly more robust than one that monitors APIs in isolation.

VII. DISCUSSION

Ethical Considerations. We contacted Microsoft and security vendors regarding our empirical results, and contacted VirusTotal about evasion results of our POC. Besides, we have communicated the recommendations of mitigation to relevant security vendors for enhanced detection mechanisms. Furthermore, we concealed key technical details in the open-source code and obscured the specific usage of APIs to mitigate security risks.

Exploration of Other Exploitable APIs. The exploitable APIs identified in our study are sourced from Microsoft

documentation. However, numerous undocumented exportable APIs exist [7]. Furthermore, we have not delved into the consistency of Native APIs corresponding to these exploitable APIs. Investigating the evasion effectiveness of direct calls to Native APIs represents a potential avenue for future research.

Cross-Platform Persistent Storage. While our research focuses on fileless attack persistent storage in Windows systems, this methodology can be extrapolated to other operating systems like Linux and macOS. Our findings provide valuable insights for researchers exploring persistent storage techniques on these platforms.

VIII. CONCLUSION

In this paper, we propose a novel method for achieving persistent storage of malicious code in fileless attacks using legitimate Windows APIs. This approach involves the use of Windows APIs to deceive antivirus software, making them perceive the execution as benign, while storing malicious code outside or indirectly within the file system. The code can be retrieved and executed as needed. With the help of rule-based classification from ChatGPT, we automated the identification of exploitable APIs. Subsequent manual verification resulted in a significant list of usable APIs. Our evaluation demonstrates that these APIs can effectively evade detection by the majority of mainstream antivirus solutions, presenting substantial security threat in real-world scenarios. Our study highlights the feasibility of using legitimate APIs for persistent storage, raising awareness in the security community for their unexpected usage.

ACKNOWLEDGMENTS

This work was supported in part by the National Key R&D Program of China (No.2023YFB3106800) and the National Natural Science Foundation of China (No.62272410).

APPENDIX

We enumerate all 64 APIs that we identified exploitable, and categorize them based on their functionalities into 9 types, including *UEFI firmware*, *cookies*, *encryption*, *Credentials*, *event logs*, *certificates*, *services*, *directly-manipulated Registry* and *others*, detailed in Table IV. For each API in these types, we provided detailed information, including the exploitable parameters, required privileges, corresponding APIs for shellcode retrieval, maximum call count, maximum payload size per API call, and the outcomes from both static and dynamic detection for the associated PoC-W and PoC-R.

Please note that the term “unlimited” means there is no restriction on the maximum call count and maximum payload size per API call. We determine the maximum payload size to be “unlimited” if there is no restriction on the payload storage capacity in a single API call for storing a malicious shellcode (typically less than 10MB). Similarly, we determine the maximum call count to be “unlimited” if there is no limit on the number of API calls when storing 1 byte each time.

It is important to clarify that “unlimited” does not imply absolute infinity. Rather, it indicates that there are no restrictions

TABLE IV: List of all identified exploitable APIs meeting criteria (italics indicate APIs that passed our evading evaluation)

API Name API Type	Description Exploited Parameter	Location Retrieval API	Novelty Required Privileges	Static Detection Result Dynamic Detection Result	Max Payload (byte) Max Call Count
<i>SetFirmwareEnvironmentVariableA(W)</i> UEFI Firmware	Sets a firmware environment variable PVOID pValue	UEFI GetFirmwareEnvironmentVariableA(W)	Reported in 2023 [19] Administrator	3/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	65535 Unlimited
<i>SetFirmwareEnvironmentVariableExA(W)</i> UEFI Firmware	Sets a firmware environment variable with the attributes PVOID pValue	UEFI GetFirmwareEnvironmentVariableExA(W)	Reported in 2023 [19] Administrator	3/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	65535 Unlimited
InternetSetCookieA(W) Cookies	Sets a cookie for a URL LPC(W)STR lpzCookieData	Database InternetGetCookieA(W)	Novel Normal	6/72, no label(PoC-W) 7/72, no label(PoC-R) 2/10 (PoC-W) 0/10 (PoC-R)	5120 Unlimited
InternetSetCookieExA(W) Cookies	Sets a cookie for a URL LPC(W)STR lpzCookieData	Database InternetGetCookieExA(W)	Novel Normal	6/72, no label(PoC-W) 7/72, no label(PoC-R) 2/10 (PoC-W) 0/10 (PoC-R)	5120 Unlimited
InternetSetCookieEx2 Cookies	Sets a cookie with more extended options INTERNETCOOKIE2 *pCookie	Database InternetGetCookieEx2	Novel Normal	5/72, no label(PoC-W) 6/72, no label(PoC-R) 3/10 (PoC-W) <i>ESET</i> and <i>etc.</i> 0/10 (PoC-R)	16383 Unlimited
CryptRegisterOIDInfo Encryption	Registers Object Identifier (OID) information PCCRYPT_OID_INFO pInfo	Registry CryptFindOIDInfo	Novel Administrator	6/72, no label(PoC-W) 5/72, no label(PoC-R) 2/10 (PoC-W) <i>Defender</i> and <i>etc.</i> 0/10 (PoC-R)	Unlimited Unlimited
<i>CryptSetOIDFunctionValue</i> Encryption	Sets the attribute for OID function const BYTE *pbValueData	Registry CryptGetOIDFunctionValue	Novel Administrator	6/72, no label(PoC-W) 5/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>NCryptCreatePersistedKey</i> Encryption	Creates a persisted cryptographic key LPCWSTR pszKeyName	Encrypted File NCryptEnumKeys	Novel Normal	7/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	255 Unlimited
<i>NCryptSetProperty</i> Encryption	Sets a property for a key storage object PBYTE pInput	Encrypted File NCryptGetProperty	Novel Normal	7/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>CryptSIPAddProvider</i> Encryption	Adds a Subject Interface Package (SIP) provider SIP_ADD_NEWPROVIDER *psNewProv	Registry RegQueryValueExW	Novel Administrator	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
CredUIStoreSSOCredW Credentials	Stores single sign-on credentials PCWSTR pszUsername	Database CredUIReadSSOCredW	Novel Normal	13/72, trojan.tedy(PoC-W) 10/72, trojan.tedy(PoC-R) 6/10 (PoC-W) 5/10 (PoC-R) <i>Kaspersky</i> and <i>etc.</i>	450 Unlimited
<i>CredWriteDomainCredentialsA(W)</i> Credentials	Writes domain credentials to the user's credential set PCREDENTIALA(W) Credential	Database CredReadDomainCredentialsA(W)	Novel Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	255 Unlimited
<i>CredWriteA(W)</i> Credentials	Creates a new credential PCREDENTIALA(W) Credential	Database CredReadA(W)	Novel Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	255 Unlimited
<i>AuthzInstallSecurityEventSource</i> Event Logs	Installs a new security event source PWSTR szEventMessageFile	Registry AuthzEnumerateSecurityEventSources	Novel Administrator	6/72, no label(PoC-W) 4/72, no label(PoC-R) 2/10 (PoC-W) <i>McAfee</i> and <i>etc.</i> 0/10 (PoC-R)	Unlimited Unlimited
ReportEventA(W) Event Logs	Reports an event to the event log LPCVOID lpRawData	EventLog File ReadEventLogA(W)	Reported in 2022 [23] Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	61440 Unlimited
<i>CertAddEncodedCR(T)lToStore</i> Certificates	Adds an encoded certificate revocation or trust list to a store const BYTE *pbCertEncoded	Registry CertFindCR(T)lInStore	Novel Normal	6/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>CertAddEncodedCertificateToStore</i> Certificates	Adds an encoded certificate to a store const BYTE *pbCertEncoded	Registry CertFindCertificateInStore	Novel Normal	8/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>CertAddCR(T)lContextToStore</i> Certificates	Adds a certificate revocation or trust list to a certificate store PCCR(T)l_CONTEXT pCr(t)lContext	Registry CertFindCR(T)lInStore	Novel Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>CertSetCertificateContextProperty</i> Certificates	Sets a property for a certificate context const void *pvData	Registry CertGetCertificateContextProperty	Novel Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>CertAddEncodedCertificateToSystemStoreA(W)</i> Certificates	Adds an encoded certificate to a system store const BYTE *pbCertEncoded	Registry CertFindCertificateInStore	Novel Normal	7/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>CertAddSerializedElementToStore</i> Certificates	Adds a serialized element to a certificate store const BYTE *pbElement	Registry CertFindCertificateInStore	Novel Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>CreateServiceA(W)</i> Services	Creates a service object LPC(W)STR lpDisplayName	Registry GetServiceDisplayNameA(W)	Novel Administrator	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	255 Unlimited
<i>SetServiceA(W)</i> Services	Registers from the registry a network service LPSERVICE_INFOA(W) lpServiceInfo	Registry GetServiceA(W)	Novel Administrator	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	255 Unlimited
<i>WSASetServiceA(W)</i> Services	Registers from the registry a network service LPWSAQUERYSETA(W) lpqRegInfo	Registry WSALookupServiceBeginA(W)	Novel Administrator	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	255 Unlimited
SHOpenRegStream2A(W) Directly-manipulated Registry	Opens a registry key as a stream LPC(W)STR pszValue	Registry SHOpenRegStream2A(W)	Novel Normal	6/72, no label(PoC-W) 4/72, no label(PoC-R) 3/10 (PoC-W) <i>Biddefender</i> and <i>etc.</i> 0/10 (PoC-R)	Unlimited Unlimited
SHOpenRegStreamA(W) Directly-manipulated Registry	Opens a registry key as a stream LPC(W)STR pszValue	Registry SHOpenRegStreamA(W)	Novel Normal	6/72, no label(PoC-W) 4/72, no label(PoC-R) 3/10 (PoC-W) <i>Biddefender</i> and <i>etc.</i> 0/10 (PoC-R)	Unlimited Unlimited
SHRegCreateUSKeyA(W) Directly-manipulated Registry	Creates a user-specific registry key LPC(W)STR pszPath	Registry SHRegEnumUSKeyA(W)	Novel Normal	7/72, no label(PoC-W) 4/72, no label(PoC-R) 1/10 (PoC-W) detected by <i>Biddefender</i> 0/10 (PoC-R)	255 Unlimited
SHRegSetPathA(W) Directly-manipulated Registry	Sets a file path in the registry LPC(W)STR pcszPath	Registry SHRegGetPathA(W)	Novel Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 3/10 (PoC-W) <i>Biddefender</i> and <i>etc.</i> 0/10 (PoC-R)	Unlimited Unlimited
SHRegSetUSValueA(W) Directly-manipulated Registry	Sets a registry subkey value in a user-specific subtree const void *pvData	Registry SHRegGetUSValueA(W)	Reported in 2014 [33] Normal	7/72, no label(PoC-W) 4/72, no label(PoC-R) 2/10 (PoC-W) <i>Biddefender</i> and <i>McAfee</i> 0/10 (PoC-R)	Unlimited Unlimited
SHRegWriteUSValueA(W) Directly-manipulated Registry	Writes a value to subkey in a user-specific subtree const void *pvData	Registry SHRegGetUSValueA(W)	Reported in 2021 [22] Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 2/10 (PoC-W) <i>Biddefender</i> and <i>McAfee</i> 0/10 (PoC-R)	Unlimited Unlimited
SHSetValueA(W) Directly-manipulated Registry	Sets the value of a registry key LPCVOID pvData	Registry SHGetValueA(W)	Novel Normal	6/72, no label(PoC-W) 4/72, no label(PoC-R) 3/10 (PoC-W) <i>Biddefender</i> and <i>etc.</i> 0/10 (PoC-R)	Unlimited Unlimited
SetupSetSourceListA(W) Directly-manipulated Registry	Sets the list of installation sources PC(W)STR *SourceList	Registry RegQueryValueExA	Novel Normal	5/72, no label(PoC-W) 5/72, no label(PoC-R) 3/10 (PoC-W) <i>Defender</i> and <i>etc.</i> 0/10 (PoC-R)	Unlimited Unlimited
<i>SetupAddToSourceListA(W)</i> Others	Appends a value to the list of installation sources PC(W)STR Source	Registry RegQueryValueExA	Novel Normal	5/72, no label(PoC-W) 6/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>WritePrivateProfileStructA(W)</i> Others	Copies data in the specified section of initialization file LPCVOID lpStruct	Encrypted File GetPrivateProfileStructA(W)	Novel Normal	5/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	32767 Unlimited
<i>SHMessageBoxCheckA(W)</i> Others	Displays a message box with a checkbox LPC(W)STR pszRegVal	Registry RegQueryValueExA	Novel Normal	4/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	16383 Unlimited
<i>SetupDiSetDeviceRegistryPropertyA(W)</i> Others	Sets a Plug and Play device property for a device const BYTE *PropertyBuffer	Registry SetupDiGetDeviceRegistryPropertyA(W)	Novel Administrator	5/72, no label(PoC-W) 7/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>SetupDiSetDevicePropertyW</i> Others	Sets a device instance property const BYTE *PropertyBuffer	Registry SetupDiGetDevicePropertyW	Novel Administrator	5/72, no label(PoC-W) 7/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	Unlimited Unlimited
<i>MFTRegister</i> Others	Adds informations of Media Foundation Transform LPWSTR pszName	Registry MFTGetInfo	Novel Administrator	4/72, no label(PoC-W) 4/72, no label(PoC-R) 0/10 (PoC-W) 0/10 (PoC-R)	80 Unlimited
WinHttpSetDefaultProxyConfiguration Others	Sets the default WinHTTP proxy configuration WINHTTP_PROXY_INFO *pProxyInfo	Registry WinHttpGetDefaultProxyConfiguration	Novel Administrator	7/72, no label(PoC-W) 6/72, no label(PoC-R) 4/10 (PoC-W) <i>Defender</i> and <i>etc.</i> 0/10 (PoC-R)	Unlimited Unlimited

imposed by the API itself. However, these limits are subject to hardware and operating system constraints. For example, the *SetFirmwareEnvironmentVariableA* function is constrained by the memory limitations of the firmware hardware itself, with the specific capacity value dependent on the

firmware hardware. For registry-related APIs, the maximum payload size is typically the maximum size of a registry entry (single-entry limit is 1,048,576 bytes under standard formats, and it equals total available memory in the latest formats).

REFERENCES

- [1] A. Afreen, M. Aslam, and S. Ahmed, "Analysis of fileless malware and its evasive behavior," in *2020 International Conference on Cyber Warfare and Security (ICCWS)*. IEEE, 2020, pp. 1–8.
- [2] O. Alexander, M. Belisle, and J. Steele, "Mitre att&ck for industrial control systems: Design and philosophy," *The MITRE Corporation: Bedford, MA, USA*, vol. 29, 2020.
- [3] F. Apap, A. Honig, S. Hershkop, and et al., "Detecting malicious software by monitoring anomalous windows registry accesses," in *Recent Advances in Intrusion Detection: 5th International Symposium, RAID 2002 Zurich, Switzerland, October 16–18, 2002 Proceedings* 5. Springer, 2002, pp. 36–53.
- [4] M. Atapattu and B. Jayawardena, "An approach to detect fileless malware that maintains persistence in windows environment," 2021.
- [5] Av-comparatives, "Summary report 2023," <https://www.av-comparatives.org/tests/summary-report-2023/>.
- [6] F. Barr-Smith, X. Ugarte-Pedrero, M. Graziano, and et al., "Survivalism: Systematic analysis of windows malware living-off-the-land," in *2021 IEEE Symposium on Security and Privacy*. IEEE, 2021, pp. 1557–1574.
- [7] U. Bayer, A. Moser, C. Kruegel, and E. Kirda, "Dynamic analysis of malicious code," *Journal in Computer Virology*, vol. 2, pp. 67–77, 2006.
- [8] S. Cesare, Y. Xiang, and W. Zhou, "Malwise—an effective and efficient classification system for packed and polymorphic malware," *IEEE Transactions on Computers*, vol. 62, no. 6, pp. 1193–1206, 2012.
- [9] K. Coogan, S. Debray, T. Kaochar, and G. Townsend, "Automatic static unpacking of malware binaries," in *2009 16th Working Conference on Reverse Engineering*. IEEE, 2009, pp. 167–176.
- [10] CrowdStrike, "Who needs malware? how adversaries use fileless attacks to evade your security," <https://www.crowdstrike.com/resources/white-papers/who-needs-malware-how-adversaries-use-fileless-attacks-to-evade-your-security/>.
- [11] S. Debray and W. Evans, "Profile-guided code compression," *ACM SIGPLAN Notices*, vol. 37, no. 5, pp. 95–105, 2002.
- [12] Y. Dehfouli and A. Habibi Lashkari, "Memory analysis for malware detection: A comprehensive survey using the oscar methodology," *ACM Computing Surveys*, 2025.
- [13] T. Dupuy and M. Faou, "Gelsemium," in *Technical Report*. ESET, 2021.
- [14] H. Fang, Y. Wu, S. Wang, and et al., "Multi-stage binary code obfuscation using improved virtual machine," in *International Conference on Information Security*. Springer, 2011, pp. 168–181.
- [15] Gartner, Inc., "Endpoint protection platforms reviews and ratings," <https://www.gartner.com/reviews/market/endpoint-protection-platforms>, 2024.
- [16] M. Graeber, "Abusing windows management instrumentation (wmi) to build a persistent, asynchronous, and fileless backdoor," *Black Hat. Las Vegas, NV, USA*, 2015.
- [17] H. Irshad, G. Ciocarlie, A. Gehani, and et al., "Trace: Enterprise-wide provenance tracking for real-time apt detection," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4363–4376, 2021.
- [18] S. Jha, S. Cui, S. Banerjee, and et al., "MI-driven malware that targets av safety," in *2020 50th annual IEEE/IFIP international conference on dependable systems and networks (DSN)*. IEEE, 2020, pp. 113–124.
- [19] jthuraismy, "Gospelroom: Data storage in uefi nvram variables," <https://gist.github.com/jthuraismy/e602d5d870230df3ce00178001f9ac16>.
- [20] Kaspersky, "securelist," <https://securelist.com/>.
- [21] Y. Kawakoya, M. Iwamura, and M. Itoh, "Memory behavior-based automatic malware unpacking in stealth debugging environment," in *2010 5th International Conference on Malicious and Unwanted Software*. IEEE, 2010, pp. 39–46.
- [22] R. Lakshmanan, "New fileless malware uses windows registry as storage to evade detection," <https://thehackernews.com/2021/12/new-fileless-malware-re-uses-windows.html>.
- [23] D. LEGEZO, "A new secret stash for "fileless" malware," <https://securelist.com/a-new-secret-stash-for-fileless-malware/106393/>.
- [24] H. Li, Y. Hao, and et al., "Enhancing static analysis for practical bug detection: An llm-integrated approach," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 474–499, 2024.
- [25] Z. Li, Q. A. Chen, C. Xiong, and et al., "Effective and light-weight deobfuscation and semantic-aware attack detection for powershell scripts," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1831–1847.
- [26] S. Liu, G. Peng, H. Zeng, and et al., "A survey on the evolution of fileless attacks and detection techniques," *Computers Security*, vol. 137, p. 103653, 2024.
- [27] Y.-F. Liu, L.-W. Zhang, J. Liang, and et al., "Detecting trojan horses based on system behavior using machine learning method," in *2010 International Conference on Machine Learning and Cybernetics*, vol. 2. IEEE, 2010, pp. 855–860.
- [28] LMArena Team, "Leaderboard Changelog: notable updates to model leaderboards," <https://news.lmarena.ai/leaderboard-changelog/>, 2025.
- [29] lockheedmartin, "cyber kill chain," <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>.
- [30] LoLBAS, "Lolbas," <https://lolbas-project.github.io/>.
- [31] Z. Lu, J. Wu, S. Zhang, and et al., "Research on ws-management-based system and network resource management middleware model," in *2009 IEEE International Conference on Web Services*. IEEE, 2009, pp. 1051–1053.

- [32] K. Maduranga, "Investigate windows management instrumentation (wmi) attacks in windows operating systems," Ph.D. dissertation, 2017.
- [33] malwaretech, "A fileless rootkit," <https://www.malwaretech.com/2014/12/phase-bot-fileless-rootkit>.
- [34] R. Meng, M. Mirchev, M. Böhme, and et al., "Large language model guided protocol fuzzing," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024.
- [35] Microsoft, "Netserversetinfo function," <https://learn.microsoft.com/en-us/windows/win32/api/lmserver/nf-lmserver-netserversetinfo>.
- [36] Microsoft, "Using wmi," <https://learn.microsoft.com/en-us/windows/win32/wmisdk/using-wmi>.
- [37] mrd0x, "malapi.io," <https://malapi.io/>.
- [38] N. Nissim, A. Cohen, and Y. Elovici, "Aldocx: detection of unknown malicious microsoft office documents using designated active learning methods based on new structural feature extraction methodology," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 3, pp. 631–646, 2016.
- [39] T. Ongun, J. W. Stokes, J. B. Or, and et al., "Living-off-the-land command detection using active learning," in *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*, 2021, pp. 442–455.
- [40] K. Oosthoek and C. Doerr, "Sok: Att&ck techniques and trends in windows malware," in *Security and Privacy in Communication Networks: 15th EAI International Conference, SecureComm 2019, Orlando, FL, USA, October 23-25, 2019, Proceedings, Part I 15*. Springer, 2019, pp. 406–425.
- [41] Openai, "Chatgpt," <https://chat.openai.com/>.
- [42] F. Pierazzi, S. Cristalli, D. Bruschi, and et al., "Glyph: Efficient ml-based detection of heap spraying attacks," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 740–755, 2020.
- [43] S. M. Pontiroli and F. R. Martinez, "The tao of. net and powershell malware analysis," in *Virus bulletin conference*, 2015, pp. 1–26.
- [44] I. V. Popov, S. K. Debray, and G. R. Andrews, "Binary obfuscation using signals," in *USENIX Security Symposium*, 2007, pp. 275–290.
- [45] K. A. Roundy and B. P. Miller, "Binary-code obfuscations in prevalent packer tools," *ACM Computing Surveys (CSUR)*, vol. 46, no. 1, pp. 1–32, 2013.
- [46] G. Rusak, A. Al-Dujaili, and U.-M. O'Reilly, "Ast-based deep learning for detecting malicious powershell," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 2276–2278.
- [47] H. Saissi, S. Winter, O. Schwahn, and et al., "Tracesanitizer - eliminating the effects of non-determinism on error propagation analysis," in *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2020, pp. 52–63.
- [48] B. Sanjay, D. Rakshith, R. Akash, and et al., "An approach to detect fileless malware and defend its evasive mechanisms," in *2018 3rd International Conference on Computational Systems and Information Technology for Sustainable Solutions*. IEEE, 2018, pp. 234–239.
- [49] R. Stamp, "Living-off-the-land abuse detection using natural language processing and supervised learning," *arXiv preprint arXiv:2208.12836*, 2022.
- [50] D. Tanana, "Behavior-based detection of cryptojacking malware," in *2020 Ural Symposium on Biomedical Engineering, Radioelectronics and Information Technology*. IEEE, 2020, pp. 0543–0545.
- [51] D. Uroz and R. J. Rodríguez, "Characteristics and detectability of windows auto-start extensibility points in memory forensics," *Digital Investigation*, vol. 28, pp. S95–S104, 2019.
- [52] S. Varlioglu, N. Elsayed, Z. ElSayed, and M. Ozer, "The dangerous combo: Fileless malware and cryptojacking," *SoutheastCon 2022*, pp. 125–132, 2022.
- [53] Veriato, "Veriato," https://help.veriato.com/products/veriato-Recon-360/Win/EN/quickstart/Deploy/Firewall_WMI.htm.
- [54] Virustotal, "Deception at scale: How malware abuses trust," <https://assets.virustotal.com/reports/2022deception.pdf>.
- [55] VirusTotal, "VirusTotal," <https://www.virustotal.com>.
- [56] D. Wael, A. Shosha, and S. G. Sayed, "Malicious vbscript detection algorithm based on data-mining techniques," in *2017 Intl Conf on Advanced Control Circuits Systems (ACCS) Systems & 2017 Intl Conf on New Paradigms in Electronics & Information Technology (PEIT)*. IEEE, 2017, pp. 112–116.
- [57] D. Wagner and P. Soto, "Mimicry attacks on host-based intrusion detection systems," in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, 2002, pp. 255–264.
- [58] Windows, "Programming reference for the win32 api," <https://learn.microsoft.com/en-us/windows/win32/api/>.
- [59] G.-W. Wong, Y.-T. Huang, Y.-R. Guo, and et al., "Attention-based api locating for malware techniques," *IEEE Transactions on Information Forensics and Security*, 2023.
- [60] L. Yang, S. Zhu, J. Shi, and et al., "Measuring and modeling the label dynamics of online anti-malware engines," in *2020 USENIX Security Symposium*, 2020.
- [61] L.-X. Yang, P. Li, Y. Zhang, and et al., "Effective repair strategy against advanced persistent threat: A differential game approach," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 7, pp. 1713–1728, 2018.
- [62] I. You and K. Yim, "Malware obfuscation techniques: A brief survey," in *2010 International conference on broadband, wireless computing, communication and applications*. IEEE, 2010, pp. 297–300.
- [63] Zenodo, "Our source code," <https://doi.org/10.5281/zenodo.13140631>.